

API Sentinel — Project Handbook & Interview Guide

Full Architecture, Functionality, Performance Engineering & Interview Q&A;

1. System Overview & Functionality (How It Works & How To Use)

API Sentinel is a comprehensive, defensive API Security, Vulnerability Monitoring, and Continuous Analytics platform designed to safeguard web services and APIs against security weaknesses, misconfigurations, and performance anomalies. The system operates strictly within user-authorized boundaries and incorporates multi-layered defensive guardrails.

How the Platform Operates (Workflow):

- **Step 1 — Target Registration & Discovery:** Users register API base URLs or import OpenAPI 3.x/Swagger JSON/YAML schemas. The parser automatically extracts endpoints, HTTP methods, query/path parameters, and authentication requirements.
- **Step 2 — Defensive SSRF Pre-Flight Verification:** Before sending network requests, the SSRF engine validates URL schemes (HTTP/HTTPS only), resolves DNS, and rigorously blocks loopbacks (127.0.0.0/8), RFC 1918 private subnets, link-local, and cloud metadata IMDS IPs (169.254.169.254).
- **Step 3 — Multi-Profile Security Assessment:** Users trigger assessments (Quick, Standard, Deep, or Custom) that dispatch 10 modular scanner plugins via asynchronous I/O with concurrency controls.
- **Step 4 — Evidence Redaction & Triage:** When a vulnerability is found, evidence is collected, cryptographic tokens and auth headers are automatically sanitized, CVSS risk scores are calculated, and findings are stored in the database.
- **Step 5 — Continuous Monitoring & Telemetry:** Periodic background probes track uptime %, latency percentiles (avg, p95), and evaluate response drift against a 24-hour moving baseline (triggering alerts at >2.5 sigma deviation).
- **Step 6 — Executive Reporting:** Instant on-demand generation of audit-ready HTML/PDF, structured JSON, and raw CSV reports mapped directly to OWASP API Top 10, CWE, and NIST controls.

2. System Outputs & Deliverables

The platform produces rich visual, analytical, and structured data across all operational views:

Output Area	Key Information & Deliverables Produced
SOC Cyber Dashboard	Top KPI stat cards (Monitored APIs, Endpoints, Active Vulns, Mean Uptime, Security Posture Score), 7-day vulnerability trend Area chart, Severity distribution Donut chart, Top Risk Targets matrix, and Recent Scans activity feed.
API Inventory Hub	Categorized API cards with environment badges (Dev, Test, Staging, Prod), security score gauges (0-100), discovered endpoint counts, open vulnerability counts, and quick-action scan launchers.
API Security Hub	Deep multi-tab view for each target: Discovered Endpoints table, Active Vulnerabilities list, 24h Response Latency Time-Series chart, Historical Scans table, and Masked Credential manager.

Vulnerability Investigation	Multi-factor Risk Score (0-10), CVSS v3.1 vector, Redacted Technical Evidence trace (headers, payload snippets), copyable cURL reproduction command, actionable remediation code, OWASP/CWE mappings, and collaborative triage comment threads.
Continuous Telemetry	24-hour response latency percentiles (avg, p95), availability % gauges, and statistical anomaly event logs with percent-deviation tracking.
Executive Reports	Audit-ready printable HTML/PDF reports, structured JSON exports for SIEM pipelines, and CSV spreadsheets detailing compliance mappings and remediation roadmaps.

3. Comprehensive Technology Stack

Layer / Component	Technologies Used & Technical Rationale
Frontend Framework	Next.js 14 (App Router) + React 18 + TypeScript: Server and client components, strict type safety, fast page transitions, and responsive layout architecture.
Styling & Aesthetics	Tailwind CSS + Lucide Icons: Curated dark SOC cybersecurity color palette (#090d16 slate-950, cyan/sky accents, glowing status indicators).
Data Visualization	Recharts: Responsive SVG Area Charts for 24h latency trends, Radial & Pie charts for severity distributions, and custom tooltip formatting.
Backend Core API	FastAPI (Python 3.11+ / 3.13): High-performance async ASGI web framework, automatic OpenAPI generation, Pydantic v2 data validation schemas, dependency injection for auth & RBAC.
Database & ORM	SQLAlchemy 2.0 (Async) + SQLite (aiosqlite) / PostgreSQL (asyncpg): Fully asynchronous non-blocking database queries, connection pooling, and declarative models.
Security & Cryptography	PyJWT + Passlib (Bcrypt) + Cryptography (AES-GCM / Fernet): Stateless JWT tokens, salted password hashing, credential encryption at rest with PBKDF2 key derivation.
Network & Scanners	HTTPX (Async Client) + dnspython: Asynchronous non-blocking HTTP probes, pre-flight DNS SSRF resolution, connection pooling, and custom timeouts.
DevOps & Containers	Docker Compose + Nginx Reverse Proxy + Pytest: Multi-container orchestration (Backend, Frontend, Demo API, PostgreSQL, Redis, Nginx) with automated unit test suites.

4. How Latency & Throughput Are Optimized

API Sentinel achieves enterprise throughput and sub-millisecond database/API overhead through careful engineering decisions:

- **1. Fully Asynchronous Event-Driven Architecture (ASGI):** Built entirely on Python's asyncio and FastAPI/Uvicorn. Incoming HTTP requests, background scanner tasks, and database queries execute concurrently without blocking the main event loop.
- **2. Async Database I/O with Connection Pooling:** Uses SQLAlchemy 2.0 async engine with asyncpg (PostgreSQL) and aiosqlite. Reusable connection pools eliminate the latency of establishing new TCP/TLS database handshakes for every transaction.

- **3. Concurrent Non-Blocking Scanner Probing:** The scanner framework dispatches probes concurrently using `asyncio.gather()` with bounded `asyncio.Semaphore` concurrency limits. This completes multi-point security audits across dozens of endpoints in 2–4 seconds instead of minutes.
- **4. Persistent HTTP Connection Keep-Alive:** Reusable `httpx.AsyncClient` connection pools reuse TCP/TLS tunnels across scanner probes, reducing round-trip latency by up to 60%.
- **5. Client-Side State & Optimistic UI:** Next.js 14 utilizes fast client routing, local state caching for telemetry series, and streamlined bundle sizes (shared chunks ~87 kB) for sub-second page loads.
- **6. Lightweight Statistical Computation:** Baseline calculations and anomaly detection compute rolling means and standard deviations in $O(N)$ memory/time using indexed database timestamps.

5. Interview Mastery: How to Explain to an Interviewer

The 60-Second Elevator Pitch:

*'I engineered **API Sentinel**, a full-stack defensive API security monitoring and vulnerability management platform. It combines automated vulnerability scanning across 10 security vectors, continuous latency and uptime telemetry with statistical anomaly detection, and enterprise RBAC. I built the backend in FastAPI using fully asynchronous SQLAlchemy and HTTPX with strict SSRF defense controls and encrypted credential vaults, and the frontend in Next.js 14 and Tailwind CSS with real-time SOC cyber telemetry charts.'*

Top Technical Interview Questions & Model Answers:

Q1: How do you prevent the scanner from becoming a vector for Server-Side Request Forgery (SSRF)?

We enforce a strict pre-flight SSRF filter before any scanner request is dispatched. The system parses the URL, restricts schemes exclusively to HTTP/HTTPS, resolves the hostname to IPv4/IPv6 addresses via DNS, and blocks loopback addresses (127.0.0.0/8), RFC 1918 private subnets (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16), link-local addresses (169.254.0.0/16), and cloud metadata IMDS IPs (169.254.169.254, metadata.google.internal). Local testing is only permitted when explicit environment flags are set.

Q2: How is sensitive data (like API tokens and evidence traces) protected across the system?

Credentials stored for authenticated scanning are encrypted at rest using AES-GCM / Fernet symmetric cryptography with keys derived via PBKDF2-HMAC. In the UI and logs, tokens are masked (e.g. sk-live-***xyz). When scanners capture request/response evidence, a regex-based sanitization pipeline automatically redacts Authorization headers, bearer tokens, API keys, and sensitive cookie headers before writing to the database.

Q3: How does your risk scoring and API Security Score algorithm work?

We use a composite scoring model. Individual vulnerabilities are assigned a base score using CVSS v3.1 metrics (Base Severity: Critical=9.5, High=8.0, Med=5.0, Low=2.5) multiplied by confidence ratings (Confirmed=1.0, Firm=0.85, Tentative=0.6) and environment exposure factors (Production=1.2, Staging=1.0, Dev=0.8). The API Security Score is a 0–100 scale computed by deducting weighted vulnerability penalties from a baseline of 100.

Q4: How did you implement statistical anomaly detection without heavy ML infrastructure?

We maintain a rolling 24-hour time-series of response latency and HTTP status codes for each API. The service computes the baseline mean (μ) and standard deviation (σ). If subsequent probe responses deviate by more than 2.5 standard deviations (Z-score > 2.5) or if the 5xx error rate exceeds 40%, the system flags an anomaly event and dispatches deduplicated alerts.

Q5: How does the platform scale under heavy scanning workloads?

The backend runs on an asynchronous ASGI event loop. Background scans run as non-blocking tasks. We use connection pooling in HTTPX and SQLAlchemy with semaphore-based rate limiting to prevent socket exhaustion and protect target APIs from unintentional Denial of Service. In production, Redis and Celery/Arq can be attached to distribute scanner workloads across worker nodes.